Universidade Federal de Mato Grosso do Sul
Faculdade de Computação

Tópicos em Computação II 2026

Paulo A. Pagliosa

PROVA 2 – Parte Prática

17/06/2026

A classe `TriangleMesh`, em `p2/TriangleMesh.h`, define uma malha de triângulos com somente dois tipos de elementos topológicos: vértices e faces (triangulares), os quais são armazenados em vetores. Os atributos de um vértice são suas coordenadas espaciais e, opcionalmente, as coordenadas da normal à superfície representada (exata ou aproximadamente) pela malha na posição do vértice. Um triângulo é caracterizado por uma tripla de inteiros iguais aos índices de seus três vértices no vetor de vértices. A estrutura é bastante compacta e suficiente para renderização (baseada em triângulos) em uma aplicação gráfica. Contudo, não é a mais adequada para operações interativas de edição ou transformação de uma malha. Por exemplo, a remoção de um vértice (ou aresta) ocasiona a remoção de todos os triângulos incidentes no vértice (ou aresta), informação esta não diretamente representada na estrutura. Outras consultas topológicas úteis em modelagem geométrica também não são disponíveis, tais como quais as arestas incidentes em um vértice, ou quais as arestas (ou faces) de borda em uma malha aberta ou com cavidades.

O objetivo geral da prova é a implementação de estrutura de dados topológica para representação poligonal de *variedades de dimensão 2*, ou apenas variedades, com ou sem borda. Informalmente, uma variedade sem borda é um espaço em que todo ponto possui uma vizinhança que se parece com o plano $\mathbb{R}^2$; com borda, os pontos de borda têm vizinhança se parece com o semi-espaço superior do $\mathbb{R}^2$, como comentado em sala. A estrutura é uma malha $M = \{\mathcal{V}, \mathcal{E}, \mathcal{F}, \mathcal{B}\}$, em que $\mathcal{V}$, $\mathcal{E}$, $\mathcal{F}$, $\mathcal{B}$ são coleções de vértices, arestas, faces e contornos, respectivamente. A estrutura é baseada no conceito de *semi-aresta*. Suponha uma aresta, $e = (v_a, v_b) \in \mathcal{E}$, com extremidades nos vértices $v_a \in \mathcal{V}$ e $v_b \in \mathcal{V}$. A "divisão" de e resulta em duas semi-arestas, h_a e h_b, com direções opostas: h_a sai de v_a e vai para v_b enquanto h_b sai de v_b e vai para v_a. As semi-arestas de uma aresta podem ser chamadas de opostas ou "gêmeas". Uma face (triangular), $f = (v_a, v_b, v_c) \in \mathcal{F}$, é definida por uma sequência fechada de semi-arestas h_a, h_b e h_c que partem dos vértices v_a, v_b e v_c, respectivamente. Se e é uma aresta interior, então e incide exatamente em duas faces, com h_a e h_b pertencendo a cada uma das duas faces incidentes na aresta. Um contorno, $b \in \mathcal{B}$, é uma sequência fechada de semi-arestas que não pertecem a uma face. Mais detalhadamente, os elementos topológicos são:

- Semi-aresta. Uma semi-aresta "conhece" o vértice da qual parte, a sua semi-aresta gêmea, a aresta a qual pertence, a face a qual pertence (que pode ser nula, caso em que a semi-aresta pertence a um contorno) e as semi-arestas posterior e anterior do laço de semi-arestas da sua face (ou contorno).

- Vértice. Um vértice tem um identificador (número inteiro não negativo) e conhece uma semi-aresta (qualquer) da qual parte. Embora não seja um atributo topológico, por simplicidade o vértice também conhece as coordenadas (x, y, z) de sua posição espacial.

- Aresta. Uma aresta tem um identificador e conhece suas duas semi-arestas gêmeas.

- Face. Uma face tem um identificador e conhece uma semi-aresta (qualquer) de seu laço de semi-arestas.

- Contorno. Um contorno tem um identificador e conhece uma semi-aresta (qualquer) de seu laço de semi-arestas.

As coleções de componentes topológicos de um malha M podem ser implementadas como listas duplamente encadeadas. Entretanto, é mais eficiente usar vetores como `std::vector`, conforme justificado em sala. Neste caso, pode-se estimar um valor inicial para o número de vértices, n_v, e adotar para o número de arestas $n_e \approx 3n_v$ e para o número de faces $n_f \approx 2n_v$. Para o número de contornos, pode-se tomar $n_b = 0$. Tais valores são usados para reserva de memória de cada vetor. (Note que se as semi-arestas gêmeas de uma aresta forem armazenadas consecutivamente no vetor de semi-arestas, o vetor de arestas pode ser dispensado.) Ainda, o identificador de um componente topológico é o índice dentro do respectivo vetor.

Como exemplo de navegação na estrutura, a função a seguir devolve o número de faces incidentes em um vértice:

```
auto counIncidentFaces(const Vertex& v)
{
  auto fc = 0;
  auto he = halfEdge(v);

  do
  {
    if (hasFace(he))
      fc++;
    he = next(twin(he));
  }
  while (he != halfEdge(v));
  return fc;
}
```

Na função acima, `halfEdge(v)` devolve a semi-aresta do vértice `v`, `hasFace(he)` verifica se a semi-aresta `he` pertence a uma face e `twin(he)` e `next(he)` devolvem a semi-aresta gêmea e a próxima semi-aresta do laço (da face ou contorno) de `he`.

Em outro exemplo, a função a seguir calcula o perímetro de um contorno:

```
auto perimeter(const Boundary& b)
{
  auto len = 0.0f;
  auto he1 = halfEdge(b);

  do
  {
    auto he2 = next(he1);

    len += distance(vertex(he1), vertex(he2));
    he1 = he2;
  }
  while (he1 != halfEdge(v));
  return len;
}
```

Similar ao exemplo anterior, `halfEdge(b)` devolve a semi-aresta do contorno `b`. Já a função `vertex(he)` devolve o vértice do qual a semi-aresta `he` parte, enquanto `distance(v1, v2)` devolve a distância euclidiana entre os vértices `v1` e `v2`.

As principais funcionalidades requeridas para a estrutura são:

- Construção a partir de um objeto do tipo `TriangleMesh`.
- Obtenção de iteradores para vértices, arestas, faces e contornos.

- Métodos de processamento, por um função qualquer, de:
  - Todas as arestas incidentes em um vértice.
  - Todas as faces do k-anel de faces de um vértice, $k > 0$. O 1-anel de faces de um vértice é o conjunto das faces incidentes no vértice. O 2-anel é o conjunto de todas as faces incidentes nos vértices das faces do 1-anel, e assim por diante.
  - Todas as faces do k-anel de faces de uma face.

Outro objetivo é estender a classe `MeshDecoration`, em `p2/MeshDecoration.h`, para decoração de vértices, arestas, faces e contornos da estrutura de dados baseada em semi-arestas, conforme discutido em sala.

Tarefas

As atividades da prova são:

A1 Definir classes para uma malha M e seus componentes topológicos.

A2 Implementar os métodos de construção, obtenção de iteradores e processamento de vizinhos conforme descrito anteriormente.

A3 Estender a classe de decoração de malhas para decoração dos componentes topológicos de um malha M.

A4 Escrever funções de teste (que devem ser chamadas na função principal em `Main.cpp`) de todas as funcionalidades de uma malha M. Além disso, proponha um "pipeline" com pelo menos 3 estágios com decorações distintas. O programa deve funcionar para malhas sem e com borda (obtenha modelos no formato Wavefront OBJ de repositórios da Internet, que podem ser lidos com a função em `p2/OBJReader.cpp`).

A5 Gravar um vídeo (de no máximo 10 minutos) em que o(s) autor(es) aparece(m) 1) explicando as definições efetuadas no código fonte compartilhado em tela e 2) mostrando a compilação e execução do programa e seus resultados.

Entrega da prova

A prova deve ser entregue via AVA em arquivo único compactado chamado `p2.zip` contendo **somente** os arquivos de código fonte do diretório `p2` e de projeto, sem quaisquer outros arquivos resultantes da compilação, bem como os arquivos Wavefront OBJ utilizados no programa. Todo arquivo `.h` e `.cpp` novo ou estendido deve ter um comentário com identificação do(s) autor(es). Se não for usado o Visual Studio, adicione um makefile para geração do executável. Em adição, `p2.zip` deve conter um arquivo texto chamado `README.txt` com o(s) nome(s) do(s) autor(es), uma breve descrição de quais atividades foram parcial ou completamente concluídas ou não e um link para o vídeo (no YouTube ou Google Drive, por exemplo).

A prova pode ser desenvolvida em grupo de no máximo três estudantes (uma submissão por grupo). Não serão considerados e terão nota zero programas em que houver detecção de plágio de qualquer que seja a fonte, mesmo que parcialmente, exceto o código do repositório da disciplina. Programas que não compilam ou não executam terão nota zero. A entrega deve ser feita até o dia e hora limites estipulados no AVA.

Lista de objetivos

A avaliação da prova levará em conta se:

- ☐ O arquivo `p2.zip` foi submetido com o conteúdo correto, incluindo `README.txt`.
- ☐ O vídeo em que o(s) autor(es) explicam o trabalho foi submetido.
- ☐ A construção de uma malha M foi implementada corretamente.
- ☐ Os iteradores de componente topológicos e os métodos de processamento de vizinhos foram implementados corretamente.
- ☐ A decoração de uma malha M foi implementada corretamente.
- ☐ Todas as funcionalidades de uma malha M e as decorações são testadas no programa.